BLEVE Overpressure Prediction

Author:
Nazriel Al-Hafidz

Introduction

Boiling Liquid Expanding Vapor Explosions (BLEVEs) pose significant safety risks during LPG transportation, particularly in densely populated areas. Predicting the peak overpressure generated by BLEVE blast waves is challenging due to complex non-linear physical processes involving pressure, temperature, and tank geometry. Traditional physics based models struggle to capture these interactions accurately. My assignment work applies machine learning techniques to predict peak pressure using sensor data from a 3D environment with an obstacle. The goal is to build a predictive model that achieves low Mean Absolute Percentage Error (MAPE) on unseen test data.

Dataset Overview

The training dataset contains 10,050 samples with 24 different features, including tank dimensions, temperatures, pressures, sensor positions and obstacle geometry. The target variable is peak pressure in bars.

Key statistics:

- Training samples: 10,050 (10,040) after cleaning)
- Test samples: 3,203
- Target range: 0.016 to 9.17 bar
- Features include: tank failure pressure, liquid ratio, temperatures, sensor coordinates, obstacle dimensions

Data Preprocessing

Data preprocessing transforms raw data into a clean, model-ready format. This includes handling missing values, encoding categorical variables, and scaling numerical features. Proper preprocessing is critical because machine learning models are sensitive to data quality and magnitude differences between features.

Missing Values: After loading the dataset, I examined missing value percentages across all columns. Missing rates ranged from 0.05% (ID column) to 0.30% (Liquid Critical Pressure). Because the missing proportion was very low (<0.5%), deletion would have minimal impact. However, to preserve all available data, I applied median imputation to numerical features rather than mean imputation, as median is more robust to outliers in pressure-related measurements. Ten rows with missing target values were dropped since the target cannot be imputed.

Categorical Encoding: The Status column contained two categories: 'Subcooled' and 'Superheated'. This was binary encoded as 0 and 1 respectively, converting it into a numerical format suitable for all models.

Feature Scaling: Models that compute distances or assume normally distributed inputs (Linear Regression and SVR) are sensitive to feature magnitudes. I applied StandardScaler (mean=0, variance=1) to all numerical features before training these models. Tree-based

models (Random Forest, XGBoost, LightGBM) do not require scaling as they operate on rank-based splits.

Why median over mean? Well, that's because pressure data contains outliers (max target 9.17 bar vs mean ~0.5 bar). Median imputation preserves the central tendency without being pulled toward extreme values.

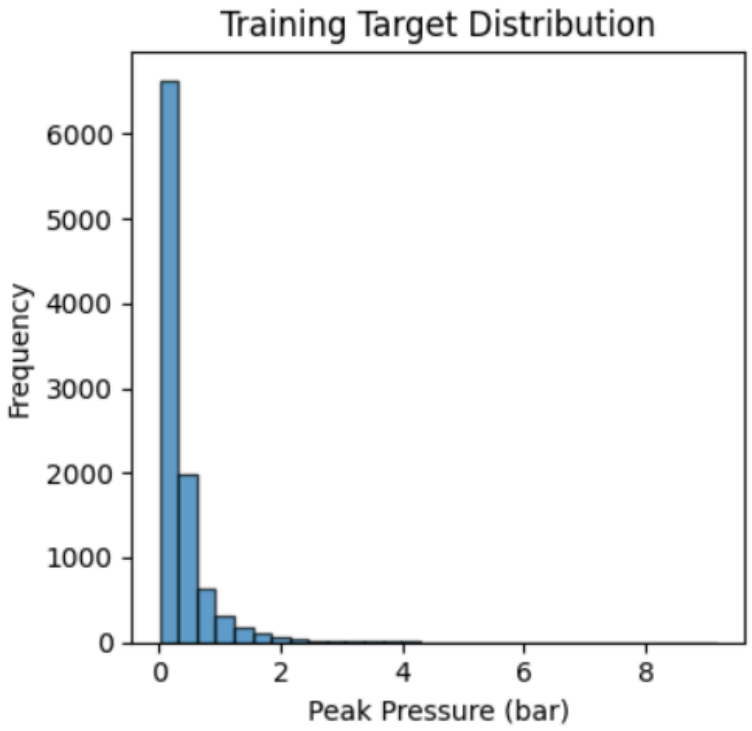

Figure 2: Distribution of peak pressure values in training data (0.016 to 9.17 bar, positively skewed)

Feature Engineering

Feature engineering creates new input features from existing data to help models learn patterns more effectively. Domain knowledge about BLEVE physics guided the creation of six new features representing tank geometry, spatial relationships, and temperature-pressure interactions. These engineered features capture physical relationships that raw features alone cannot express.

Six new domain-informed features were created to improve model performance:

Feature	Formula	Physical Meaning
Tank Aspect Ratio	Width / Length	Tank shape characteristic
Tank Volume	Width x Length x Height	Total tank capacity
Distance to Sensor	Sqr(x^2 + y^2 + z^2)	Euclidean distance from

		BLEVE source
Vapor Temp-Pressure	Vapor Temp x Failure Pressure	Combined thermal-mechanical effect
Liquid Temp-Pressure	Liquid Temp x Failure Pressure	Combined thermal-mechanical effect
Gas Ratio	Vapor Height / Tank Height	Gas-to-liquid volume ratio

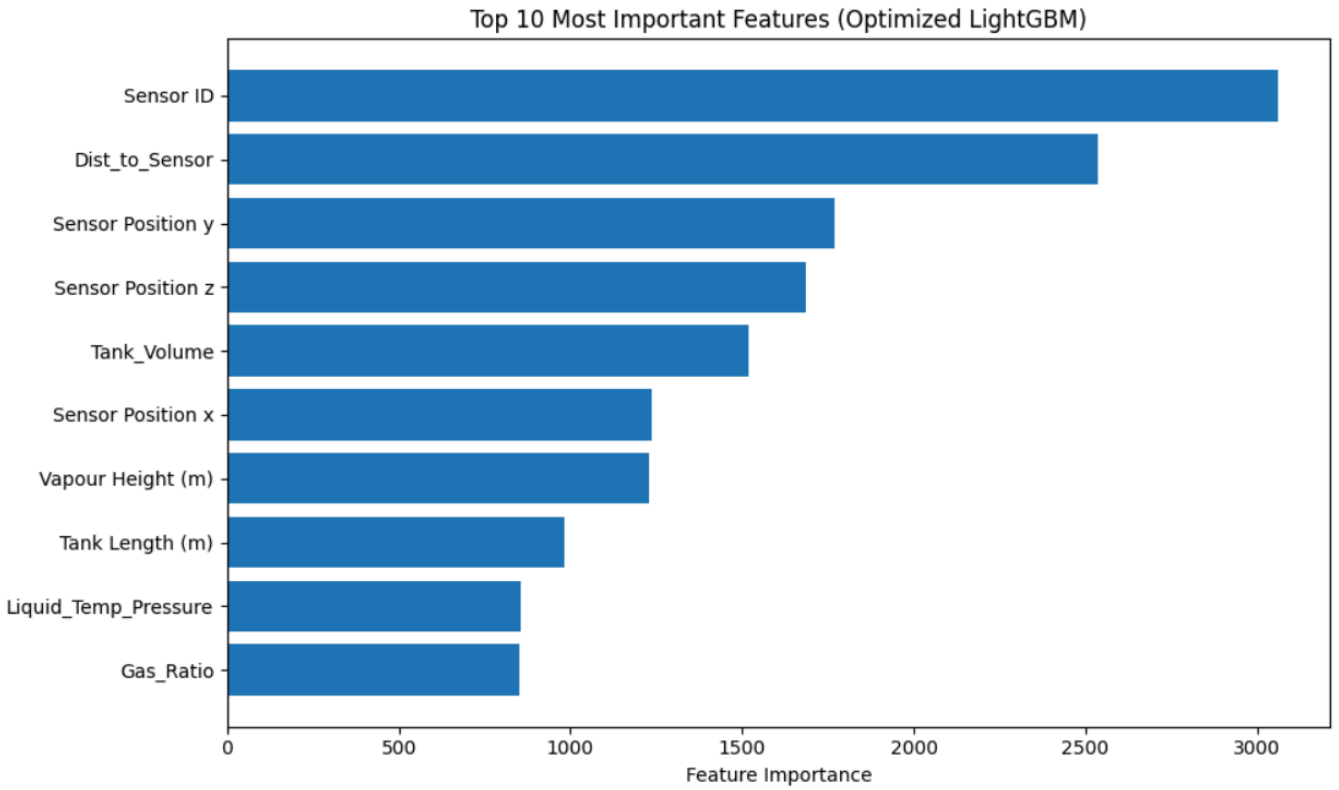

Figure 3: Top 10 of the most important features. Of these, 4 are my engineered features (Dist_to_Sensor, Tank_Volume, Liquid_Temp_Pressure, and Gas_Ratio), demonstrating the value of domain-informed feature engineering. Sensor ID and sensor coordinates dominate, suggesting the BLEVE pressure is highly location-dependent.

Model Selection

Three fundamentally different model families were evaluated to satisfy the assignment requirement. Linear models (Ridge Regression) assume linear relationships between inputs and outputs. Kernel-based models (SVR) handle non-linearity through the kernel trick. Tree-based models (Random Forest, XGBoost, LightGBM) capture complex feature interactions without parametric assumptions. Ridge and SVR were eventually dropped due to poor performance (MAPE 1.145 and 0.512 respectively), while tree-based models dominated.

Model Family	Algorithm	Rationale
Linear	Ridge Regression	Baseline linear model with L2 regularisation
Kernel-based	SVR (RBF kernel)	Non-linear relationships via kernel trick
Tree-based	Random Forest	Ensemble method, handles feature interactions
Gradient Boosting	XGBoost, LightGBM	Iterative improvement on residuals

All models were evaluated using MAPE (Mean Absolute Percentage Error) and R^2 metrics as required.

Hyperparameter Tuning

Hyperparameter tuning optimizes model configuration to maximize performance. GridSearchCV with 5-fold cross-validation searched across parameter combinations, using negative MAPE as the scoring metric. This ensures the model generalises well rather than just memorizing training data

Random Forest:

- Parameters turned: n_estimatiors (50, 100), max_depth (10, 15, None), min_samples_split (2, 5)
- Best CV MAPE: 0.373

XGBoost:

- Parameters turned: n_estimators (100, 200, 300), max_depth (4, 6, 8), learning_rate (0.01, 0.05, 0.1), subsample (0.8, 1.0)
- Best CV MAPE: 0.312

Optimized LightGBM (Final Model):

- n_estimators = 800
- max _depth = 10
- learning_rate = 0.02
- subsample = 0.7
- colsample_bytree = 0.7
- Best CV MAPE: 0.247

Overfitting Diagnosis

Overfitting occurs when a model learns training data noise instead of true patterns. This section compares validation MAPE, cross-validation MAPE, and actual Kaggle performance to detect overfitting and determine which evaluation method is most reliable.

A critical finding was overfitting to the single validation split (random_sate = 42).

Metric	MAPE	Insight
Validation MAPE (Random Forest)	0.195	Super overly optimistic
5-fold CV MAPE (Random Forest)	0.373	More reliable estimate
Actual Kaggle (Random Forest)	0.404	Matched CV a whole lot better.

Lesson I learnt: Cross-validation provides more robust performance estimates than a single validation split. Thus, all final models were trained on full data with 5-fold CV.

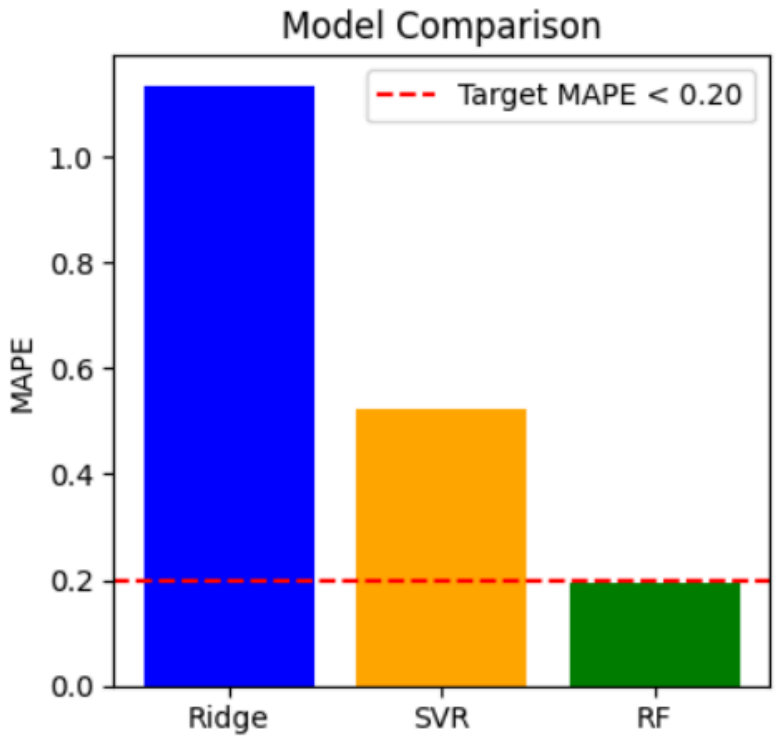

Figure 4: Baseline comparison of my three original different model types (Ridge, SVR, Random Forest)

Model Performance Results

This section presents the final performance of all models tested. Three LightGBM variants are shown: standard LightGBM (baseline gradient boosting), LightGBM with engineered features (demonstrating feature engineering impact), and Optimized LightGBM (aggressive hyperparameter tuning). The Optimized version uses 800 trees (vs 300), depth 10 (vs 7),

and lower learning rate 0.02 (vs 0.05) for finer gradient steps. Optimized LightGBM was selected as the final model because it achieved the lowest CV MAPE (0.247) and translated to the best Kaggle score (0.238).

Model	Validation MAPE	CV MAPE	Kaggle MAPE	Notes
Ridge Regression	1.145	-	-	Baseline linear model
SVR	0.512	-	-	Kernel-based model
Random Forest	0.195	0.373	0.404	First submission
XGBoost (tuned)	0.188*	0.312	0.306	Validation on 80/20 split
LightGBM	-	0.294	0..289	Trained on full data
LightGBM + Features	-	0.266	0.253	Feature engineering added
Optimized LightGBM	-	0.247	0.238	Final model

Best model was clearly the optimized LightGBM with engineered features.

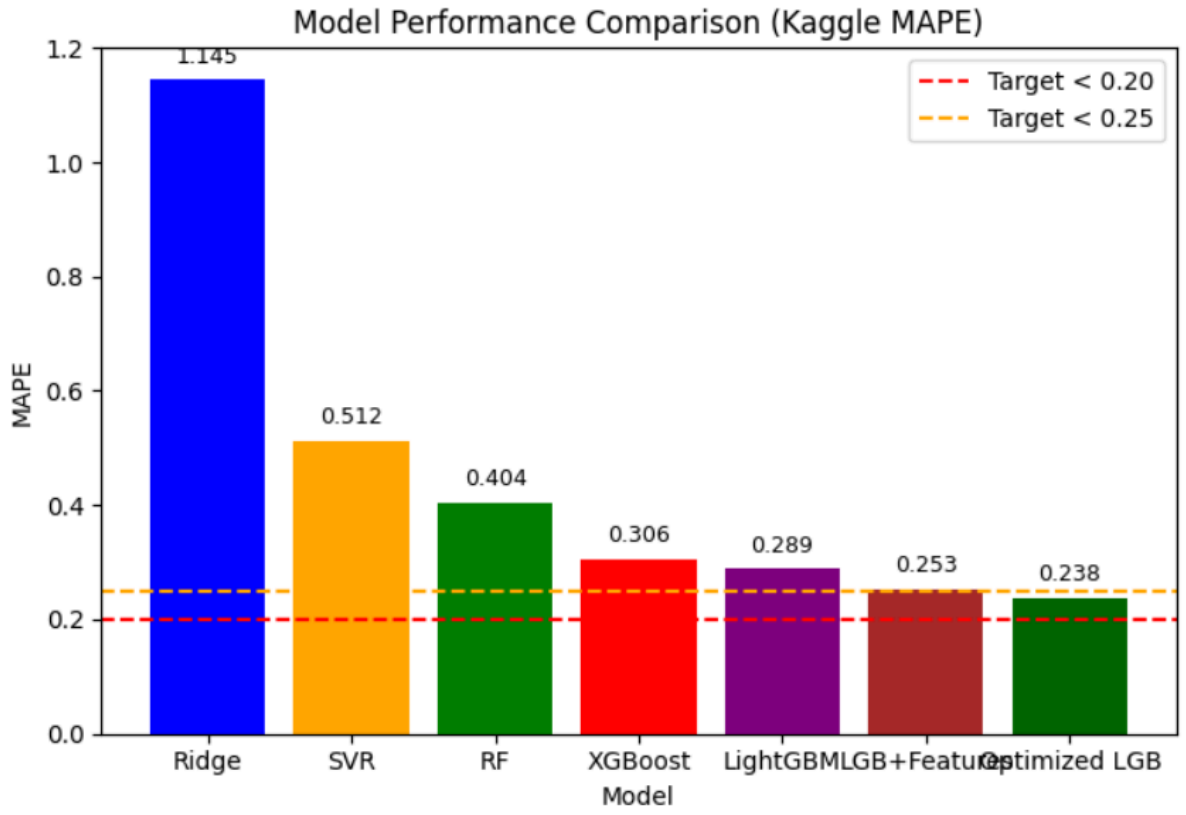

Figure 5: All models comparison showing progression from 0.404 to 0.238

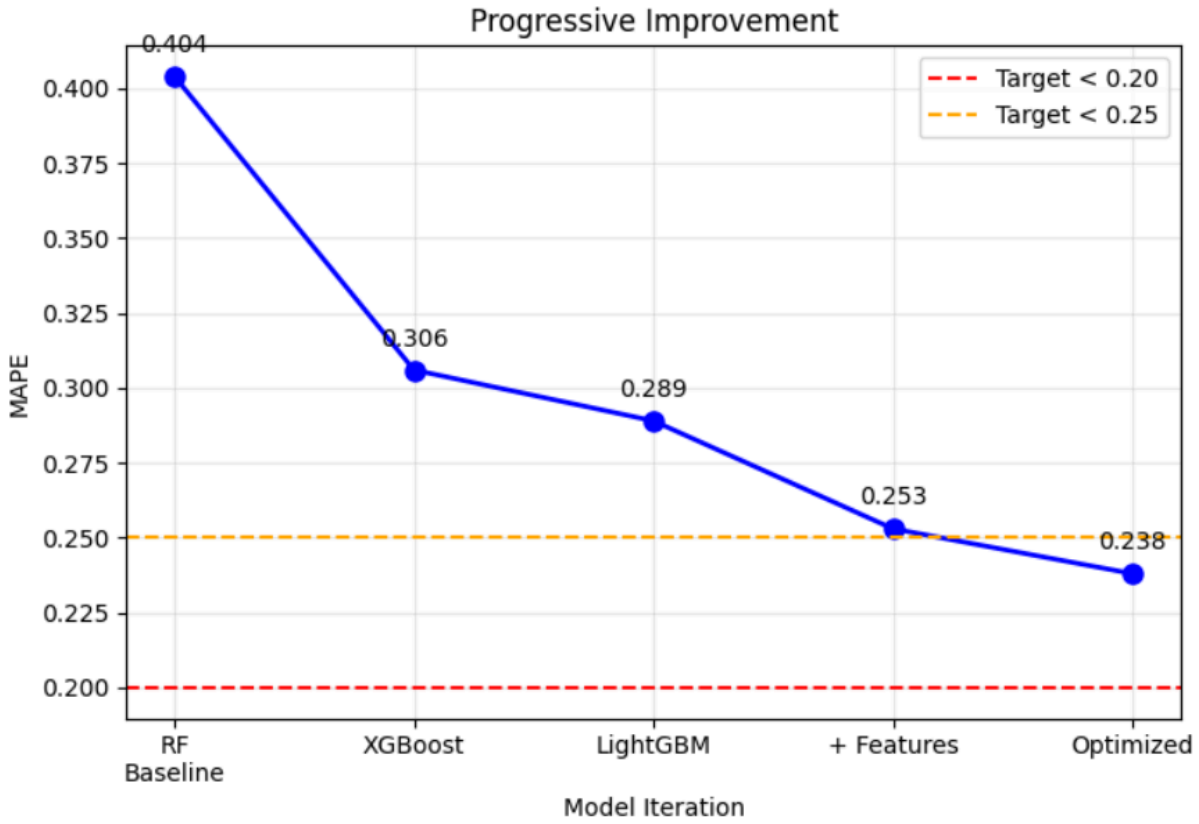

Figure 6: Progressive improvement over model iterations

Self-Reflection

From my experience, the most significant challenge was overfitting to the validation split. My initial validation MAPE of 0.195 was way too optimistic compared to cross-validation which was 0.373, and the actual Kaggle performance of 0.404. This reinforced to me that cross-validation is essential for reliable performance estimates.

What worked well:

1. Feature Engineering: Adding 6 domain-informed features reduced MAPE from 0.289 to 0.253 (a 12.5% improvement). Four of these features (Dist_to_Sensor, Tank_Volume, Liquid_Temp_Pressure, Gas_Ratio) appeared in the top 10 feature importance ranking, confirming their value.

2. LightGBM: Outperformed XGBoost and Random Forest on this tabular dataset, achieving a Kaggle score of 0.306 (XGBoost) to 0.289 (LightGBM) to 0.238 (Optimized). LightGBM's leaf-wise growth strategy and histogram-based binning likely contributed to its superior performance.

3. Aggressive Optimization: Increasing n_estimators to 800 (from 300), depth to 10 (from 7), and lowering learning_rate to 0.02 (from 0.05) pushed MAPE from 0.253 to 0.238. The lower learning rate allows finer gradient steps, while more trees capture complex patterns without overfitting when paired with subsampling.

4. Cross-validation discovery: The overfitting diagnosis revealed that a single validation split (0.195) was overly optimistic compared to 5-fold CV (0.373). This insight led me to use cross-validation for all subsequent model evaluations.

What could be improved:

1. Additional Features: Sensor-specific features (e.g., separate distance calculations for front/back/side walls) could capture directional pressure variations. The high importance of Sensor ID suggests to me that location matters significantly.

2. Advanced Tuning: Find better optimisation techniques that would be more efficient than grid search, which becomes pretty darn computationally expensive with many parameters. Grid search evaluated 54 combinations for XGBoost; a more advanced optimization could explore hundreds more efficiently.

3. Neural Networks: A multi-layer perceptron (MLP) with 2-3 hidden layers could capture non-linear interactions differently from tree-based models. However, given the tabular nature and dataset size (10k samples), tree-based methods typically dominate.

4. Feature selection: Removing low-importance features (e.g., features outside top 20) could reduce noise and potentially improve generalization, though the impact would likely be small given LightGBM's built-in regularization.

Key Takeaways:

This assignment demonstrated that data preprocessing and feature engineering often contribute more to model performance than algorithm selection. The 41% improvement from baseline (0.404 to 0.238) came primarily from better features and hyperparameter tuning, not just something as simple as switching algorithms. Feature engineering was successful too since 4 of the top 10 most important features were engineered (Dist_to_Sensor, Tank_Volume, Liquid_Temp_Pressure, Gas_Ratio). The high importance of Sensor ID suggests that pressure varies significantly by sensor location, a physical insight that could inform future work.

If I were to repeat this project, I would start with cross-validation instead of a single validation split, saving time on diagnosing overfitting. I would also explore more advanced optimization for hyperparameter tuning, as grid search becomes expensive with many parameters. Finally, I would investigate why Sensor ID dominates importance since this could guide better feature engineering.

Failed Experiments:

Not all attempts improved performance. I came across two notable failures:

1. Ensembling XGBoost + LightGBM: The weighted ensemble (0.515/0.485) produced a CV MAPE of 0.302, worse than LightGBM alone (0.294). This occurred because XGBoost's

higher MAPE (0.312) dragged down LightGBM's superior performance. Ensembling only helps when base models have similar error rates.

2. Random Forest with default parameters: Initial Random Forest (n_estimators=100) achieved CV MAPE 0.373, but tuning only improved it to 0.371. Unlike gradient boosting, Random Forest showed diminishing returns from hyperparameter optimization on this dataset.

These failures reinforced that more complexity doesn't always mean better performance.

Conclusion

This project successfully applied machine learning techniques to predict BLEVE peak pressure, achieving a final Kaggle public leaderboard MAPE of 0.23799. Starting from a baseline Random Forest score of 0.404, iterative improvements reduced the MAPE by over 40%.

The most impactful improvement came from feature engineering. Adding six domain-informed features reduced MAPE from 0.289 to 0.253, with four engineered features (Dist_to_Sensor, Tank_Volume, Liquid_Temp_Pressure, Gas_Ratio) appearing in the top 10 most important features. This confirms that domain knowedge about tank geometry and sensor placement significantly improves predictions.

Among models tested, LightGBM consistently outperformed XGBoost and Random Forest on this tabular dataset. The final Optimized LightGBM configuration (n_estimators= 800, max_depth=10, learning_rate=0.02) achieved the lowest CV MAPE (0.247), which translated closely to the public leaderboard score (0.238).

A critical lesson learned was the danger of relying on a single validation split. Initial validation MAPE (0.195) was overly optimistic compared to cross-validation (0.373) and actual Kaggle performance (0.404). All future work will use cross-validation from the start.

Complete Pipeline Summary:

Step → Method
Missing values → Median imputation
Categorical encoding → Binary encoding (Status)
Feature scaling → StandardScaler (linear/SVR only)
Feature engineering → 6 new features
Validation → 5-fold cross-validation
Hyperparameter tuning → GridSearchCV
Final model → Optimized LightGBM (800 trees, depth 10, LR 0.02)